Guía de Sustentación

🎓 Guía de Sustentación Técnica

Sistema Académico con Microservicios

Alumno: [Tu nombre] **Profesor:** Alejandro Riaño **Materia:** Programación Orientada a Objetos

---

📋 Índice de la Sustentación

1. Arquitectura General
2. Principios SOLID
3. Patrones de Diseño
4. Demostración Técnica
5. Preguntas Frecuentes

---

1. Arquitectura General

Diagrama de Componentes

```
Cliente (Postman) → API Gateway (:8080) → MS-Estudiantes (:8081)
                                        → MS-Cursos (:8082)
                                        → MS-Matrículas (:8083)
```

Puntos Clave a Mencionar

- **4 microservicios independientes** con bases de datos separadas (H2)
- **API Gateway** como punto único de entrada (Spring Cloud Gateway)
- **Comunicación síncrona** vía REST entre servicios
- **Módulo común** para compartir excepciones y utilidades

Stack Tecnológico

Componente	Tecnología
Lenguaje	Java 21 LTS
Framework	Spring Boot 3.2.3
Gateway	Spring Cloud Gateway
Persistencia	Spring Data JPA + H2
Mapeo	MapStruct 1.5.5
Testing	JUnit 5 + Mockito

---

2. Principios SOLID

Single Responsibility Principle (SRP)

> "Una clase debe tener una única razón para cambiar"

Ejemplo en el proyecto:

```
// StudentController - Solo maneja HTTP
@RestController
public class StudentController {
    @PostMapping
    public ResponseEntity<StudentResponse> createStudent(...) { }
}

// StudentService - Solo lógica de negocio
public class StudentServiceImpl {
    public StudentResponse createStudent(StudentCreateRequest request)
        { }
}

// StudentRepository - Solo acceso a datos
public interface StudentRepository extends JpaRepository<Student,
        Long> { }
```

Dónde mostrarlo:

- `ms-estudiantes/src/main/java/com/academic/estudiantes/`

Open/Closed Principle (OCP)

> "Abierto para extensión, cerrado para modificación"

Ejemplo en el proyecto:

```java
// Para agregar una nueva validación, NO modifico
        EnrollmentServiceImpl
// Solo creo una nueva clase:
@Component
@Order(10)
public class NewValidation implements EnrollmentValidationStrategy {
    @Override
    public void validate(EnrollmentCreateRequest request,
                         StudentResponse student,
                         CourseResponse course) {
        // Nueva lógica
    }
}
```

Dónde mostrarlo:

- `ms-matriculas/src/main/java/com/academic/matriculas/validation/`

Liskov Substitution Principle (LSP)

> "Los objetos de una clase derivada deben poder sustituir a objetos de la clase base"

Ejemplo en el proyecto:

```java
// Cualquier BusinessException puede usarse donde se espera
        BaseException
public class BusinessException extends BaseException { }
public class ConflictException extends BaseException { }

// En GlobalExceptionHandler:
@ExceptionHandler(BaseException.class)
public ResponseEntity<ErrorResponse> handleBaseException(BaseException
        ex) {
    // Funciona con cualquier subclase
}
```

Interface Segregation Principle (ISP)

> "Los clientes no deben depender de interfaces que no usan"

Ejemplo en el proyecto:

```java
// Interfaces pequeñas y específicas
public interface EnrollmentValidationStrategy {
    void validate(EnrollmentCreateRequest request,
                  StudentResponse student,
                  CourseResponse course);
    int getOrder();
}
```

```
// NO tenemos interfaces con 20 métodos
```

Dependency Inversion Principle (DIP)

> "Depender de abstracciones, no de implementaciones"

Ejemplo en el proyecto:

```
@Service
public class EnrollmentServiceImpl {
    // Depende de la INTERFAZ, no de implementaciones concretas
    private final List<EnrollmentValidationStrategy> validations;

    // Spring inyecta TODAS las implementaciones automáticamente
    public EnrollmentServiceImpl(List<EnrollmentValidationStrategy>
        validations) {
        this.validations = validations;
    }
}
```

---

3. Patrones de Diseño

Strategy Pattern

Problema que resuelve: Diferentes validaciones para crear una matrícula, cada una con su propia lógica.

Estructura:

```
EnrollmentValidationStrategy (interface)
       ↑
       ├── StudentActiveValidation    (verifica estudiante activo)
       ├── CourseActiveValidation     (verifica curso activo)
       ├── CourseCapacityValidation   (verifica cupo disponible)
       └── DuplicateEnrollmentValidation (verifica no duplicado)
```

Código clave:

```
// Interface
public interface EnrollmentValidationStrategy {
    void validate(EnrollmentCreateRequest request,
                  StudentResponse student,
                  CourseResponse course);
    int getOrder();
}

// Implementación
```

```java
@Component
public class StudentActiveValidation implements
        EnrollmentValidationStrategy {
    @Override
    public void validate(...) {
        if (!"ACTIVE".equals(student.getStatus())) {
            throw BusinessException.studentInactive(student.getId());
        }
    }

    @Override
    public int getOrder() { return 2; }
}
```

Dónde mostrarlo:

- `ms-matriculas/src/main/java/com/academic/matriculas/validation/`

Template Method Pattern

Problema que resuelve: Definir el esqueleto del algoritmo de creación de matrícula, permitiendo que subclases redefinan pasos.

Estructura del algoritmo:

```
createEnrollment()
    │
    ├── 1. fetchStudentData()      ← Obtener datos externos
    ├── 2. fetchCourseData()       ← Obtener datos externos
    ├── 3. executeValidations()    ← Ejecutar strategies
    ├── 4. createAndSaveEnrollment() ← Persistir
    └── 5. postProcessEnrollment() ← Hook para extensión
```

Código clave:

```java
@Override
@Transactional
public EnrollmentResponse createEnrollment(EnrollmentCreateRequest
        request) {
    // Template Method: pasos en orden definido
    StudentResponse student =
        fetchStudentData(request.getStudentId());
    CourseResponse course = fetchCourseData(request.getCourseId());

    executeValidations(request, student, course);

    Enrollment enrollment = createAndSaveEnrollment(request);

    postProcessEnrollment(enrollment);

    return enrollmentMapper.toResponse(enrollment);
}
```

Dónde mostrarlo:

- `ms-matriculas/src/main/java/com/academic/matriculas/service/impl/EnrollmentServiceImpl.java`

---

4. Demostración Técnica

Comandos para ejecutar

```
# 1. Compilar todo
mvn clean install -DskipTests

# 2. Ejecutar tests (con cobertura)
mvn test

# 3. Levantar servicios (orden recomendado)
# Terminal 1:
cd ms-estudiantes && mvn spring-boot:run

# Terminal 2:
cd ms-cursos && mvn spring-boot:run

# Terminal 3:
cd ms-matriculas && mvn spring-boot:run

# Terminal 4:
cd api-gateway && mvn spring-boot:run
```

Flujo de demostración con Postman

1. **Crear estudiante:**

   ```
   POST http://localhost:8080/api/v1/students
   {"firstName": "Juan", "lastName": "Pérez", "email": "juan@uni.edu"}
   ```

2. **Crear curso:**

   ```
   POST http://localhost:8080/api/v1/courses
   {"name": "POO", "description": "Curso de POO", "capacity": 30}
   ```

3. **Crear matrícula (exitosa):**

   ```
   POST http://localhost:8080/api/v1/enrollments
   {"studentId": 1, "courseId": 1}
   ```

4. **Demostrar validación (intento duplicado):**

   ```
   POST http://localhost:8080/api/v1/enrollments
   {"studentId": 1, "courseId": 1}
   ```

```
→ 409 Conflict: "ENROLLMENT_CONFLICT"
```

5. **Desactivar estudiante y demostrar validación:**

```
PATCH http://localhost:8080/api/v1/students/1/status
{"status": "INACTIVE"}

POST http://localhost:8080/api/v1/enrollments
{"studentId": 1, "courseId": 2}
→ 422: "STUDENT_INACTIVE"
```

---

5. Preguntas Frecuentes

¿Por qué microservicios y no monolito?

- **Escalabilidad:** Cada servicio escala independientemente
- **Desacoplamiento:** Cambios en uno no afectan a otros
- **Tecnología:** Cada servicio podría usar diferentes tecnologías
- **Demostración académica:** Muestra comunicación entre servicios

¿Por qué H2 y no MySQL/PostgreSQL?

- **Simplicidad:** No requiere instalación externa
- **Portabilidad:** Funciona en cualquier máquina
- **Alcance:** Para demostración académica es suficiente
- **Cambio fácil:** Solo cambiar configuración en `application.yml`

¿Por qué Spring Cloud Gateway?

- **Centralización:** Un único punto de entrada
- **Cross-cutting concerns:** Logging, correlación, CORS en un solo lugar
- **Ruteo:** Fácil configuración de rutas a microservicios

¿Cómo funciona la inyección de validaciones?

```java
// Spring detecta TODOS los beans que implementan la interfaz
// y los inyecta como lista automáticamente
@RequiredArgsConstructor
public class EnrollmentServiceImpl {
    private final List<EnrollmentValidationStrategy> validations;
    // Spring inyecta: [StudentActive, CourseActive, CourseCapacity,
        Duplicate]
}
```

¿Cómo se ordenan las validaciones?

```java
// Cada validación define su orden
public int getOrder() { return 2; } // StudentActiveValidation
```

```java
public int getOrder() { return 4; } // CourseActiveValidation
public int getOrder() { return 5; } // CourseCapacityValidation
public int getOrder() { return 6; } // DuplicateEnrollmentValidation

// En el servicio se ordenan antes de ejecutar
validations.stream()
    .sorted(Comparator.comparingInt(EnrollmentValidationStrategy::getOrder))
    .forEach(v -> v.validate(request, student, course));
```

📊 Métricas del Proyecto

Métrica	Valor
Módulos	5 (common, 3 microservicios, gateway)
Tests unitarios	19
Tests integración	17
Total tests	36
Cobertura (services)	~95%
Patrones implementados	2 (Strategy, Template Method)
Principios SOLID	5/5 verificados

📁 Archivos Clave para Mostrar

1. **Strategy Pattern:**
   - `ms-matriculas/src/main/java/com/academic/matriculas/validation/EnrollmentValidationStrategy.java`
   - `ms-matriculas/src/main/java/com/academic/matriculas/validation/impl/StudentActiveValidation.java`
2. **Template Method:**
   - `ms-matriculas/src/main/java/com/academic/matriculas/service/impl/EnrollmentServiceImpl.java`
3. **Excepciones:**
   - `common/src/main/java/com/academic/common/exception/BaseException.java`
   - `common/src/main/java/com/academic/common/exception/GlobalExceptionHandler.java`
4. **Tests:**
   - `ms-matriculas/src/test/java/com/academic/matriculas/service/EnrollmentServiceTest.java`

¡Éxito en tu sustentación! 🎓